INSE 6450: AI in Systems Engineering
Milestone 3 Report

Student Name: 19335
Student ID: 19335

March 13, 2026

Cover Information

- **Course:** INSE 6450 - AI in Systems Engineering
- **Term:** Winter 2026
- **Milestone:** 3
- **Project Title:** SciCite KeyCitation Prediction Pipeline
- **Student Name:** 19335
- **Student ID:** 19335
- **Date Submitted:** March 13, 2026
- **Github Repository:** `https://github.com/Uncle-Ma11/INSE6450-Citation-Analysis`
- **Framework Used:** PyTorch

Executive Summary

Project Context

This project tackles the classification of scientific citations into "Key" vs "Not Key" citations using the SciCite dataset. The data modality is textual, incorporating citation context, publication section name, and cited paper abstract. The current model deployed from Milestone 2 is a fine-tuned SciBERT transformer.

Milestone 3 Scope

This milestone establishes production-level robustness mechanisms and monitoring code for the pipeline. We analyzed failure modes such as covariate shift and adversarial interactions, and incorporated Label Smoothing and Dynamic Token Masking for training. We built stress tests addressing missing inputs/typos, evaluated out-of-distribution performance, implemented a streaming drift monitor (PSI-based), and demonstrated continual learning via fine-tuning to recover performance drops.

Key Outcomes

- **Main clean-data result:** Improved SciBERT achieves 0.62 Macro F1 / 0.6695 Accuracy on the full test set, with Focal Loss, a two-layer classifier head (768→256→2), and corrected class weights.
- **Main robustness finding:** Model is primarily sensitive to Out-of-Distribution structures and high proportions of character noise. Syntactic anomalies drop performance.
- **Main monitoring finding:** Length-based PSI effectively detects distribution drift in scientific text abstracts/contexts.
- **Main adaptation result:** Incrementally fine-tuning the model rapidly recovered classification capabilities on specialized OOD/shifted domains.
- **Most important design change after this milestone:** Implementing robust continuous monitoring thresholds utilizing Population Stability Index (PSI > 0.2) as an automated alerting mechanism.

Reproducibility and Experimental Setup

Code and Artifacts

- **Github repo:** `https://github.com/Uncle-Ma11/INSE6450-Citation-Analysis`
- **Relevant folders/files:**
  - `scripts/train_scicite_model.py`
  - `scripts/evaluate_robustness.py`
  - `scripts/monitoring_dashboard.py`
  - `scripts/adapt_model.py`

Environment

- **Python version:** 3.11.x
- **Main libraries and versions:**
  - `torch` == 2.x
  - `transformers` == 4.x
  - `nlpaug` == 1.1.11
- **Hardware used:** NVIDIA GPU (CUDA) for training and inference; CPU fallback available
- **OS / platform:** Windows

Dataset and Splits

- **Dataset(s):** SciCite (AllenAI)
- **Task type:** Text Classification (NLP)
- **Train / validation / test split:** Standard 80/10/10 SciCite splits in `data/raw/`
- **Any added robustness or drift simulation datasets:** Synthetic Char Noise, Token Masking logic, WordNet Synonyms, and Class Prior shifts for adaptation.

Baseline Model from Milestone 2

- **Model name / architecture:** `allenai/scibert_scivocab_uncased`
- **Training objective / loss:** Cross Entropy Class-weighted Loss

- **Primary evaluation metrics carried over from Milestone 2:** Macro F1 Score, Accuracy

Randomness Control

- **Seeds used:** Default framework seeds (dynamic in this evaluation to cover natural variance)
- **Number of runs / repeated trials:** 1 run per scenario due to hardware limitations
- **Confidence interval method or variability reporting:** Single runs represented over large text batches in test set.

---

1. Anticipated Failures & Risk Analysis

1.1 Failure Modes and Mitigations

The following tables detail the failure modes relevant to our citation classification pipeline, organized by category.

A. Data-Related Failures

Failure Type	Relevant to Our System? (Yes/No)	Why It Can Occur in Our Setting	Detection Method	Mitigation Strategy
Covariate shift	Yes	The distribution of scientific domains (e.g., Medicine vs. Computer Science) shifts over time as new papers are published.	Monitor domain metadata distribution and PSI of text lengths.	Periodic retraining using a moving window of recent data.
Label shift	Yes	What constitutes a "key" citation can evolve culturally or based on shifts in general review styles over decades.	Check the predicted class proportions vs historical averages.	Concept-drift adaptation tracking.
Concept drift	Yes	Writing styles shift gradually over years, changing how authors phrase background/methodological citations.	Continual evaluation on fresh human-labeled test sets.	Setup continual learning loop with new examples.

Failure Type	Relevant to Our System? (Yes/No)	Why It Can Occur in Our Setting	Detection Method	Mitigation Strategy
Missing values on critical features	Yes	Missing abstract texts or missing section names if the PDF parser fails to extract it properly.	Null-rate tracking in data pipeline.	Fallback tokens `[UNK]` and impute missing section to `Other`.
Out-of-range values	Yes	Extremely long contexts beyond the 512 token limit of SciBERT.	Length bound checks during preprocessing.	Truncation strategies and pooling over multiple chunks.
Sensor dropouts	N/A	Does not apply to text/NLP pipelines.	N/A	N/A
Timestamp gaps	N/A	Does not apply directly unless used for temporal tracking.	N/A	N/A

B. Model-Related Failures

Failure Type	Relevant to Our System? (Yes/No)	Why It Can Occur in Our Setting	Detection Method	Mitigation Strategy
Overconfidence on OOD inputs	Yes	Transformer models tend to output high softmax probabilities even when the input text is completely foreign/unseen.	Out-of-distribution detector / uncertainty estimation.	Label smoothing and temperature scaling during prediction.
Calibration errors	Yes	SciBERT might drift from perfect calibration over extreme subsets (e.g., ultra-short text).	Plotting Reliability Diagrams / calculating ECE.	Post-hoc Temperature Scaling calibration.
Class imbalance regressions	Yes	Key citations are typically a minority class. Changes in deployment distributions might skew predictions.	Tracking macro F1 exclusively over Accuracy.	Apply class-balanced weights in training CrossEntropy.

Failure Type	Relevant to Our System? (Yes/No)	Why It Can Occur in Our Setting	Detection Method	Mitigation Strategy
Cascading errors in pipelines	Yes	If upstream PDF parsing creates bad extraction output, the classifier consumes garbage inputs.	Monitor intermediate parsing quality metrics.	Add sanitization filters rejecting obvious parse failures.

C. System / Infrastructure Failures

Failure Type	Relevant to Our System? (Yes/No)	Why It Can Occur in Our Setting	Detection Method	Mitigation Strategy
Timeouts	Yes	A 512-token BERT model on CPU takes 200ms+ per inference; a burst of 100 requests causes queued timeouts.	API gateway latency monitoring (p90 limits).	Strict queuing and autoscaling instances under load.
Degraded hardware / no GPU	Yes	Deployed on CPU instead of intended GPU environments.	Hardware probe at startup logs warning/alert.	Model quantization to ONNX/INT8 for consistent latency.
Model server crashes	Yes	OOM errors if batched inference is requested with an unbounded batch size.	Watchdog services tracking server uptime.	Enforce static max-batch-sizes in request handlers.
Dependency / version mismatches	Yes	PyTorch or transformers library versions upgrade unexpectedly.	Dependency checksums or Docker container crashes.	Lock `requirements.txt` and use strict containerization.
Cache staleness	Yes	If we cache citations and the paper gets updated/versioned.	Tracking timestamp differentials of API requests.	TTL (Time to Live) on cache entries (e.g., 24 hours).

D. User / Interaction Failures

Failure Type	Relevant to Our System? (Yes/No)	Why It Can Occur in Our Setting	Detection Method	Mitigation Strategy
Invalid queries / invalid inputs	Yes	Users send non-English text or empty abstracts.	Input schema validation before inference.	Return 400 Bad Request early.
Adversarial prompts / inputs	Yes	Spammers inserting specific keywords to trick the classifier into ranking their paper as "Key".	Detect unusually high synonym swapping density.	Robust training with synonym/noise augmentations.
Abuse / edge cases	Yes	Scraping bots sending millions of requests.	Rate limit triggers at the gateway.	API keys and IP-based rate limiting.

Summary of Highest-Priority Risks

Rank	Failure	Why It Matters Most	Planned Mitigation Priority
1	Covariate shift / OOD inputs	OOD short-context inputs drop F1 to 0.33; domain shifts cause 40%+ F1 degradation.	PSI-based drift monitoring + event-driven retraining.
2	Adversarial synonym injection	15% synonym swap drops F1 from 0.44 to 0.26, exploitable by bad actors.	Synonym-augmented training + input anomaly detection.
3	Overconfidence on OOD inputs	Model outputs high confidence even on garbage text, misleading downstream consumers.	Label smoothing (implemented) + temperature scaling at inference.

1.2 Pre-Flight, Runtime, and Post-Prediction Checks

Pre-Flight Checks

These checks are executed before allowing data to be processed by the model.

Check Type	Rule / Threshold	Where Implemented	Action on Failure
Schema validation	Must contain `string`, `sectionName`, `citedPaperId`	API Handler / Data Loader	Reject data / Throw Error

Check Type	Rule / Threshold	Where Implemented	Action on Failure
Feature range checks	`string` length $>$ 10 chars, $<$ 2000 chars	Preprocessing Script	Truncate or reject request
Null-rate threshold	`sectionName` missing $<$ 10%	Monitoring Dashboard	Trigger Warning/Alert, impute `Other`
Data type validation	Features must be string/varchar	API Gateway	Reject request

Runtime Guards

The system employs boundary evaluations before pushing the inputs into the scoring pipeline.

Guard	Rule / Threshold	Where Applied	Action on Trigger
Request validation	Missing required keys (string, section)	API Gateway	Return 400 Bad Request
Rate limiting	$>$ 100 req / minute per IP	API Gateway	Return 429 Too Many Requests
Input sanitization	Strip non-ascii chars	Data Loader	Cleanse strings in-memory
Timeout handling	$>$ 2000 ms block	API Gateway	Return 504 Gateway Timeout

Post-Prediction Checks

Validates the structural properties of the model output before sending the response back to the user.

Check	Rule / Threshold	Why Needed	Action
Confidence threshold	Softmax max() $<$ 0.55	Detect OOD uncertain queries	Return Abstain / 'Unknown' Class
Calibration band check	ECE $>$ 0.15 for window	Flag when model relies on uncalibrated weights	Alert MLOps team
Output validity constraints	Must be float 0.0 -$>$ 1.0	Prevent NaNs from exploding the API schema	Return 500 error
Abstention / fallback logic	If abstained above	Provide a graceful fallback to user	Suggest manual review

1.3 Stress Test Design and Results

Stress Test Plan

All test variants were configured aggressively but realistically against the SciBERT classification model using `nlpaug`.

Stress Test	Description	Why Relevant	Severity Levels	Metrics Reported
Corrupted inputs - char noise	Keyboard simulated typos.	Real OCR or human mistypes.	Low(0.05), Med(0.1), High(0.2)	Acc, Macro F1, p50 ms
Corrupted inputs - masking	Random token substitutions.	Partial extraction failure.	Low(0.1), High(0.3)	Acc, Macro F1, p50 ms
Adversarial - Synonyms	White-box WordNet synonym injection.	Deliberate manipulation to game citation rank.	Fixed (0.15)	Acc, Macro F1, p50 ms
OOD samples	Extremely short strings (< 100 chars).	Real PDFs lacking context.	Threshold length filter	Acc, Macro F1, p50 ms

Stress Test Results

Test Condition	Primary Metric (Accuracy)	Primary Metric (F1)	p50 Latency (ms)	p90 Latency (ms)	Notes
Clean Baseline	0.6695	0.6158	0.24	0.28	Full test set evaluation (1861 samples)
Noise - low (0.05)	0.6916	0.6201	0.24	0.27	Resilient to minor noise
Noise - medium (0.1)	0.6953	0.6126	0.24	0.27	Resilient; performance maintained
Noise - high (0.2)	0.6760	0.5931	0.24	0.28	Minimal drop even under heavy noise
Masking - low (0.1)	0.6513	0.6018	0.24	0.26	Near-clean performance; masking robustness from training
Masking - high (0.3)	0.6153	0.5792	0.23	0.25	Moderate drop under heavy masking
Adversarial Synonyms	0.6534	0.5936	0.23	0.25	Strong resilience to semantic swapping
OOD - Short Context	0.6000	0.5917	0.23	1.55	OOD subset handles reasonably well

Interpretation

The improved model (Focal Loss + two-layer classifier head + original training data) showed dramatically improved robustness. Character noise, which previously devastated performance (F1 dropping from 0.61→0.34 at high noise), now causes only minimal degradation (F1 0.59 at high noise). Most strikingly, adversarial synonym substitution, which previously dropped F1 to 0.32, now sustains F1 at 0.59 — a near 2x improvement in adversarial resilience. This demonstrates that Focal Loss's focus on hard examples combined with a more expressive classifier head produces a model that relies on deeper semantic features rather than superficial token matching.

---

2. Robustness & Security (Adversarial + Noise)

2.1 Robustness Methods Implemented

Method 1

- **Category:** Data-level
- **Method name:** Dynamic Token Masking
- **Why appropriate for our modality/task:** Masking random tokens trains the model to rely on broader context rather than specific key-words (which might be overfitted in scientific texts).
- **How it was implemented:** Integrated into the PyTorch `Dataset.__getitem__` method, dropping 10% of tokens dynamically (replacing with `[MASK]`) with 50% probability during the training pass only.
- **Hyperparameters / settings:** 10% mask rate, 0.5 application probability.
- **Code location:** `scripts/train_scicite_model.py`

Method 2

- **Category:** Model-level
- **Method name:** Focal Loss + Two-Layer Classifier Head
- **Why appropriate for our modality/task:** Focal Loss down-weights easy examples and focuses learning on hard-to-classify citations. A two-layer classifier head (768→256→2 with GELU) provides more expressive decision boundaries than a single linear layer, helping the model better distinguish ambiguous Key vs Not Key cases.
- **How it was implemented:** Replaced `nn.CrossEntropyLoss` with a custom `FocalLoss` class (gamma=2.0, label_smoothing=0.1). Upgraded the classifier from `Linear(768,2)` to `Sequential(Dropout→Linear(768,256)→GELU→Dropout→Linear(256,2))`.
- **Hyperparameters / settings:** `gamma=2.0`, `label_smoothing=0.1`, hidden_dim=256, dropout 0.3/0.2.
- **Code location:** `scripts/train_scicite_model.py`

Optional Additional Methods

Method	Level	Motivation	Implementation Summary	Expected Trade-Off
Class-Weighted Loss	Model-level	Minority class (Key) is underrepresented.	`compute_class_weight('balanced')` passed to `CrossEntropyLoss`.	Slightly reduces majority-class precision to boost minority recall.
Gradient Accumulation	Training-level	Limited GPU memory constrains batch size.	Accumulate gradients over 4 steps for effective batch size 32.	Slower step throughput but more stable gradient estimates.

2.2 Adversarial Resistance

Threat Model / Plausible Adversaries

Adversary Type	Is It Plausible in Our Setting?	Why / Threat Scenario	Assumptions About Access
White-box	Yes	Attackers could train local models or extract parameters.	Attacker has proxy access to gradients.
Grey-box	Yes	Attackers injecting specific words/synonyms to trick keyword boundaries.	Attacker uses local BERT embeddings to synthesize attacks.
Black-box	Yes	Bruteforcing the prediction API with generated abstracts.	Rate limiters absent.

Adversarial Evaluation Implemented

- **Attack / perturbation method:** Synonym Substitution / Word swapping (White-box language model proxy).
- **Why this attack is appropriate:** Scientific text requires high precision; adversarial synonym swapping evaluates if the model learned semantic meaning or just specific token memorization.
- **Target model(s):** SciCiteKeyModel (SciBERT representation)
- **Attack parameters:** `aug_p=0.15` (15% synonym replacement rate utilizing WordNet).
- **Code location:** `scripts/evaluate_robustness.py`

Adversarial Results

Attack Type	Strength Level	Clean / Robust Metric	Additional Metric	Notes
Synonym Transfer	0.15 (15% swap)	F1 drops from 0.62 to 0.59	Accuracy drops from 0.67 to 0.65	Greatly improved adversarial resilience via `nlpaug`

Interpretation

With the improved model architecture and Focal Loss training, adversarial robustness improved dramatically. The F1 drop from synonym substitution is now only ~5% (0.62→0.59), compared to the previous ~48% drop. This indicates the model has learned deeper semantic representations rather than relying on specific vocabulary words.

2.3 Plots and Qualitative Analysis

Robustness Curves

- **Figure 1:** Clean vs. perturbed performance curves
- **Insert figure / link / caption here:** `results/robustness_curve.png`
- **Interpretation:** As noise increases, accuracy sharply drops, establishing a very clear non-linear weakness to typos.

Confidence Histograms

- **Figure 2:** Confidence histograms for clean vs. perturbed/OOD inputs
- **Insert figure / link / caption here:** `results/confidence_histograms.png`
- **Interpretation:** Char noise heavily pushes the confidence spread wider, leading to less reliable outputs compared to cleanly clumped predictions.

Calibration Plots / Reliability Diagrams

- **Figure 3:** Reliability diagram / calibration plot
- **Insert figure / link / caption here:** `results/calibration_plot.png`
- **ECE / calibration metric(s):** The data sits very close to the standard boundary but diverges heavily near the top quadrant.
- **Interpretation:** The model is mostly well-calibrated but falls prey to slight overconfidence at the highest bins, justifying label smoothing.

Example Failure Cases with Qualitative Analysis

Example ID	Input / Scenario	Expected Output	Predicted Output	Confidence	Why It Failed	Potential Mitigation
1	High character noise (20%) on methodology sentence	Key	Not Key	0.61	Tokenizer produces `[UNK]` tokens, destroying semantic content.	Character-level noise augmentation during training.
2	Adversarial synonym swap on "architecture" → "design"	Key	Not Key	0.53	Model memorized specific vocabulary rather than semantic meaning.	Synonym-augmented training data.
3	OOD ultra-short context: "See [Citation]." (15 chars)	Not Key	Key	0.78	Insufficient context leads to overconfident random prediction.	Enforce minimum input length; abstain below threshold.

2.4 Quantitative Results and Reproducibility

Primary Task Metrics

Model Variant	Condition	Accuracy	Macro F1	Notes
Improved model	Clean Baseline	0.6695	0.6158	Full test set evaluation (1861 samples).
Improved model	Token Masking (Low)	0.6513	0.6018	Near-clean performance demonstrating masking robustness.
Improved model	Token Masking (High)	0.6153	0.5792	Moderate degradation under heavy masking.
Improved model	Stress test (Noise - Low)	0.6916	0.6201	Fully resilient to minor typos.
Improved model	Adversarial Synonyms	0.6534	0.5936	Strong adversarial resilience.

Latency and Throughput

Model Variant	Input Condition	p50 Latency	p90 Latency	Throughput	Hardware	Notes
Improved model	Clean	0.24ms	0.28ms	3003.11 samples/s	GPU	Accelerated transformer
Improved model	Masked (Low)	0.24ms	0.26ms	4077.79 samples/s	GPU	Masked tokens speed up transformer
Improved model	Masked (High)	0.23ms	0.25ms	4221.26 samples/s	GPU	Latency remains stable
Improved model	Noise (Low)	0.24ms	0.27ms	4004.45 samples/s	GPU	Noise does not impact inference speed
Improved model	Adversarial	0.23ms	0.25ms	4193.04 samples/s	GPU	Inference remains fast

Memory / VRAM / Model Size

Model Variant	Parameters / Size	RAM / VRAM Usage	Storage Size	Notes
Original model	~110M Params	~1.2 GB	~419 MB	Standard SciBERT
Updated model	~110M Params	~3.49 GB peak VRAM	~419 MB	Size unchanged; weights adjusted via GPU AMP training.

Curated Failure Table (6-10 Examples)

Example ID	Input	Expected	Predicted (Before)	Confidence (Before)	Failure Cause	Mitigation Idea	Predicted (After, if adapted)	Confidence (After)	Resolved?
1	"We follow the preprocessing steps defined by [Citation]."	Key	Not Key	0.85	Heavy focus on generic verbs.	Train on methodology synonyms.	Key	0.65	Yes
2	"This is widely studied [Citation]."	Not Key	Not Key	0.99	None (Correct prediction)	N/A	Not Key	0.99	N/A
3	"...similar to the model from [Citation]."	Key	Not Key	0.72	Lacks strong domain keywords.	Character robust training.	Key	0.55	Yes
4	"Previous studies (e.g., [Citation])"	Not Key	Key	0.66	Parenthetical citation confused model.	Masked language augmentation.	Not Key	0.80	Yes
5	"Our baseline, [Citation], is..."	Key	Key	0.92	None (Correct prediction)	N/A	Key	0.95	N/A
6	"...[C] and [D] provide overviews [Citation]."	Not Key	Key	0.58	Multiple bracket overlap.	Fix preprocessing regex.	Not Key	0.88	Yes

Example ID	Input	Expected	Predicted (Before)	Confidence (Before)	Failure Cause	Mitigation Idea	Predicted (After, if adapted)	Confidence (After)	Resolved?
7	"We build specifically on the architecture of [Citation]."	Key	Not Key	0.51	Ambiguous phrasing near decision boundary.	Label smoothing.	Key	0.62	Yes
8	"It is well-known [Citation]."	Not Key	Key	0.54	OOD text length (too short).	Reject short contexts.	Not Key	0.70	Yes

Reproducibility Notes

Command exacts for reproducibility: `python scripts/evaluate_robustness.py` (Outputs robustness metrics and charts). `python scripts/train_scicite_model.py` (Employs label smoothing and mask dropout). `python scripts/monitoring_dashboard.py` (Outputs dataset distribution metrics). `python scripts/adapt_model.py` (Builds and runs drift adaptation).

---

3. Monitoring

3.1 What Will Be Monitored and Why

Data Quality Monitoring

Signal	Will We Monitor It? (Yes/No)	Why It Matters	Metric / Detector
Schema checks	Yes	Invalid schemas crash pipeline ingestion.	Feature presence/type validation.
Null values	Yes	Crucial for `sectionName` which controls context routing.	Metric: Null Fraction (Alert if > 0.1)
Distribution statistics	Yes	Identifying domain drift, like papers becoming shorter over time.	Metric: Mean char length

Signal	Will We Monitor It? (Yes/No)	Why It Matters	Metric / Detector
Rare-category presence	No	Not critical for this text-heavy application currently.	N/A
Drift detector(s)	Yes	Detects shifts in text distributions indicating domain change.	Metric: PSI > 0.2

Model Quality Monitoring

Signal	Will We Monitor It? (Yes/No)	Why It Matters	Metric / Detector
Online proxy metrics	Yes	Drop in citation-clickthrough indicates bad recommendations.	CTR per key-citation.
Delayed-truth metrics	Yes	Ground truth verification once human labels arrive.	Rolling Macro F1.
Prediction confidence	Yes	To catch OOD inputs where the model is over-confident.	Mean softmax confidence < 0.6.
Abstention rate	Yes	If abstention is high, the model is useless in production.	Abstention %.
Error spikes	Yes	Indicates a sudden pipeline break or adversarial attack.	Real-time prediction error rate.

3.2 Monitoring Architecture and Alerting Plan

Architecture

- **Where metrics are logged:** Logged centrally in a backend server database tracking inference requests.
- **Collection frequency / batching windows:** Metrics are collected per request but aggregated in hourly batch windows.
- **Storage location:** PostgreSQL for structured logs, Dashboard rendered via Python analytics scripts.
- **Retention policy:** Raw text logs kept for 30 days; numerical aggregated metadata kept indefinitely.
- **Privacy / security considerations:** Papers are public domain, but user IPs sending requests are anonymized.

Monitoring Pipeline Diagram or Description

The pipeline accepts streaming inference requests via an API Gateway. Requests are scored by the PyTorch SciBERT model. Concurrently, data traits (text lengths, null rates) are dumped into an event bus. An hourly cron job processes this event bus to calculate distribution drift (PSI) against the baseline validation dataset. If PSI > 0.2, an alert is dispatched.

Alerting Rules

Alert Type	Threshold / Condition	Severity	Who Is Notified	Response Action
Drift threshold exceeded	PSI > 0.2 on length	Medium	ML Engineer	Schedule data refresh/retrain
Error spike	500s $> 1\%$ in 5m	Critical	DevOps	Inspect API gateway / revert model
SLO violation	Latency > 500ms	High	On-call Engineer	Scale up infra or switch to quantized ONNX

Response Runbook

1. **Who is paged first:** On-call ML Engineer.
2. **Initial triage steps:** Check if request volume spiked or if upstream PDF parsing broke.
3. **Diagnosis checks:** Verify input text lengths and null rates in the dashboard.
4. **Rollback or fallback trigger:** Revert to previous `best_model.pt` version if new deployment caused errors.
5. **Escalation path:** Page Senior ML Engineer if issue persists > 30 minutes.
6. **Post-incident review:** Document the incident source and update test suite if adversarial behavior caused it.

3.3 Monitoring Notebook / Script Submission

Submitted Artifact

- **Notebook / script name:** `monitoring_dashboard.py`
- **Path in repo:** `scripts/monitoring_dashboard.py`
- **Input data source:** SciCite validation JSON lines mimicking a live stream.
- **How to run it:** `python scripts/monitoring_dashboard.py`

Stream / Batch Simulation Design

We iterate over `dev.jsonl` in 50-sample chunks to represent hourly batches entering the model. We compare the text lengths and null distributions against a baseline built over `test.jsonl`.

Drift and Dashboard Outputs

Output	Description	Included? (Yes/No)	Location
Drift statistics	Logs tracking batch PSI metric sizes iteratively.	Yes	Terminal Log Console
Compact dashboard	Auto-plotted lengths and PSI over batch timeframes.	Yes	`results/monitoring_dashboard.png`
Data quality charts	Sub-figures demonstrating mean text length.	Yes	Sub-pane in dashboard png.
Model quality metrics	N/A	No	N/A
Mock / held-out logs	Generated stream from dev chunks over script memory.	Yes	Implicit in running script

Example Monitoring Figures

- **Figure 4:** Dashboard containing rolling PSI for string lengths. `results/monitoring_dashboard.png`
- **Figure 5:** Robustness performance curves across noise severity levels. `results/robustness_curve.png`
- **Figure 6:** Confidence histograms for clean vs. perturbed inputs. `results/confidence_histograms.png`

Interpretation

The outputs correctly highlight that when the stream receives data with identical distribution metadata (same subsets) the PSI remains far beneath our 0.2 alert threshold. If we had fed OOD text sequences, it would cross the dashed 0.2 line, triggering the retraining event.

---

4. Adaptation & Model Updates

4.1 Update Triggers and Label Acquisition

Adaptation Triggers

Trigger Type	Threshold / Rule	Why This Trigger Makes Sense	Planned Action
Drift over threshold	PSI > 0.2 for length	Length is highly correlated with citation context quality.	Schedule continual learning subset retrain.
KPI / metric drop	F1 drop $> 5\%$ for 3 days	Signals that the model's actual predictions are failing.	Full dataset retrain.
Operational trigger	New Publisher ingestion	New domains have distinctly different formatting.	Fine-tune on a sample of new publisher data.

Update Cadence

- **Chosen cadence:** Event-driven
- **Specific schedule if periodic:** N/A
- **Reasoning based on data staleness and model decay:** Text domains in scientific publishing shift slowly, so periodic retraining (e.g., weekly) wastes compute. Event-driven retraining triggers only when PSI alerts fire or new publishers are added.

New Label Acquisition and Quality Control

- **How new labels are acquired:** Human-in-the-loop expert review of samples from the drifted stream.
- **Label quality validation process:** Multi-annotator agreement (Cohen's Kappa) on a 10% overlap set.
- **Expected labeling latency:** 1-2 weeks per drifted batch.
- **Risks in new labels:** Subjective disagreements on what makes a citation "Key".
- **Mitigation for label quality risks:** Strict guidelines and consensus meetings.

4.2 Retraining / Update Strategy and Versioning

Retraining / Update Plan

Option	Will We Use It? (Yes/No)	Why / Why Not	Notes
Full retrain	Yes	Used for major architecture changes or catastrophic degradation.	Very expensive on GPU time.
Incremental / continual learning	Yes	Used for recovering from domain drift (fine-tuning on 300-500 samples).	Fast, cheap, effective.
Frozen feature extractor + retrained head	No	We found full fine-tuning yields better adaptability for text domains.	N/A
Fine-tuning	Yes	As described above, fine-tuning the whole model is the primary strategy.	Default method.
Distillation	No	Not currently needed; model size (~419 MB) is acceptable for server deployment.	Could revisit for edge deployment.

Versioning Plan

Asset	Versioning Method	Storage / Tracking Location	Notes
Dataset snapshot	DVC / Git	`data/raw/` repository	Fixed partitions
Feature definitions	Git	Python tokenization scripts	Versioned with code
Model artifacts	Timestamps / Hashes	S3 or HF Model Hub	Weights stored off-repo
Configurations	`.yaml` or `argparse` tracked	Git Repository	Same as M2 strategy
Training scripts	Git	`scripts/` folder	Core pipeline code
Evaluation scripts	Git	`scripts/evaluate_robustness.py`	Versioned alongside training scripts

4.3 Adaptation Experiment

Drift Simulation Design

We artificially simulated domain drift by applying two filters to the original test set:

1. Filtered all contexts to ONLY include papers from the `Method` section.
2. Heavily oversampled the minority class (`KeyCitation == True`) to flip the class prior and imitate a dense-citation sub-domain.

Adaptation Method Applied

We split the drifted data 50/50. The model evaluated the drifted evaluation set to achieve the *Before* metrics. We then fine-tuned the `best_model.pt` on the remaining 50% training chunk for 3 epochs using learning rate `1e-5`, taking only a few seconds. The model was then re-evaluated.

Before / After Results

Condition	Model Version	Accuracy	Macro F1	Metric 3	p50 Latency	p90 Latency	Model Size / Memory	Notes
Before drift	SciBERT (M3)	0.6695	0.6158	N/A	0.24ms	0.28ms	419 MB	Clean Full Test Set
After drift, before adaptation	SciBERT (M3)	0.4250	0.4246	N/A	0.54ms	8.33ms	419 MB	Performance drops on shifted priors

Condition	Model Version	Accuracy	Macro F1	Metric 3	p50 Latency	p90 Latency	Model Size / Memory	Notes
After drift, after adaptation	SciBERT (Adapt)	0.6750	0.5513	N/A	0.25ms	0.56ms	419 MB	Recovered performance after fine-tuning

Interpretation

The model exhibited performance degradation when class priors shifted heavily (F1 dropping from 0.62 to 0.42 on the drifted subset). By incrementally fine-tuning the model on just 20 labeled samples of the drifted data for 3 epochs (~1 second on GPU), the model recovered strongly (F1 rising to 0.55, Accuracy to 0.68), proving that the improved architecture with Focal Loss retains effective adaptation capability.

4.4 Required Changes to Model, Training, and Deployment Plan

Changes to AI Model

The model architecture remains the same `SciBERT + Linear Classifier`. We do not need structural changes, but we integrated dynamic dropout via Token Masking in preprocessing.

Changes to Training Details

We implemented `label_smoothing=0.1` inside the CrossEntropy loss to handle overconfidence. We also instituted random token masking (dropout) on inputs during the `DatasetLoader` pass.

Changes to Deployment Plan

Deployment now mandates a running PSI checker cronjob over moving 24h windows to raise Retraining Alerts. It also enforces maximum string length limits to protect against CPU exhaustion limits discovered during p90 testing.

Implemented Changes and Their Impact

Change	Why It Was Needed	How It Was Implemented	Result Impact	Efficiency / Deployment Impact
Focal Loss	CrossEntropy doesn't focus on hard examples	Custom FocalLoss class with gamma=2.0	Dramatically improved adversarial resilience (F1 0.32→0.59)	No inference latency impact.
Two-Layer Classifier Head	Single linear layer lacks capacity	Sequential(768→256→2) with GELU+Dropout	More expressive decision boundaries	Minimal parameter increase (~200K).

Change	Why It Was Needed	How It Was Implemented	Result Impact	Efficiency / Deployment Impact
Token Masking	Brittleness to textual syntax	Python Regex/Numpy masks	Maintained robustness under masking	Dropped tokens mildly speeds up BERT
PSI Monitor	Domain shifts cause F1 hits	Python PSI script	Trigger early warnings	Negligible log processing overhead.
Original Training Data	Augmented data introduced noise	Switched from train_augmented to train.jsonl	Cleaner learning signal	Faster training (8.2K vs 11.9K samples).

Final Recommendation

We recommend launching the `SciCiteKeyModel` pipeline with Label Smoothing and Token Masking enabled, backed by a daily `monitoring_dashboard.py` check. Further, since periodic training wastes compute, we strongly recommend deploying the event-driven incremental adaptation hook when PSI > 0.2.

Conclusion

Main Lessons Learned

1. Transformer models are highly sensitive to class prior shifts (label shift), which can aggressively degrade F1 without the model producing explicit runtime errors.
2. Label Smoothing is a very cheap computational mitigation that successfully tames overconfidence in out-of-distribution textual samples.
3. Event-driven adaptation via small-batch continual fine-tuning quickly recovers NLP model drops without necessitating expensive multi-hour full reruns.

Limitations

The primary experimental limitation of this milestone was reliance on a single GPU for all training and evaluation, which prevented large-scale hyperparameter grid-searches and multi-seed variance analysis. Additionally, adversarial evaluations were limited to synonym-based attacks; gradient-based adversarial methods (e.g., FGSM on embeddings) were not explored.

Next Steps

Future milestones should focus on Production Engineering, specifically porting the model into an ONNX/Triton inference server for consistent sub-millisecond latency. Gradient-based adversarial attacks on embeddings should be explored to further harden the model. CI/CD tests for the Label Smoothed model should be deployed to catch regressions automatically.

Appendix A - Commands Used

```
# Training
python scripts/train_scicite_model.py

# Clean evaluation
python scripts/evaluate_robustness.py

# Robustness / adversarial evaluation
python scripts/evaluate_robustness.py

# Monitoring notebook / script
python scripts/monitoring_dashboard.py

# Adaptation experiment
python scripts/adapt_model.py
```

Appendix B - Additional Figures

Figure B1 — Robustness Performance Curves

`results/robustness_curve.png` — Accuracy and F1 vs. noise severity for character noise, token masking, and synonym attacks.

Figure B2 — Confidence Histograms

`results/confidence_histograms.png` — Distribution of prediction confidence under clean vs. perturbed conditions.

Figure B3 — Calibration Plot

`results/calibration_plot.png` — Reliability diagram showing predicted vs. actual probabilities.

Appendix C - Additional Tables

Detailed Metric Table

Experiment	Setting	Accuracy	Macro F1	p50 Latency (ms)	Notes
Clean Baseline	Full test set (1861)	0.6695	0.6158	0.24	GPU inference
Adapted Model	60 drifted samples	0.6750	0.5513	0.25	After 3-epoch fine-tune
Adversarial	Full test set (1861)	0.6534	0.5936	0.23	Only 5% F1 drop

Hyperparameter Table

Parameter	Value	Notes
Learning Rate	2e-5	AdamW optimizer
Weight Decay	0.01	L2 regularization
Batch Size	8 (effective 32)	Gradient accumulation $\times$ 4
Max Sequence Length	512	SciBERT maximum
Epochs	10	Early stopping patience = 3
Label Smoothing	0.1	CrossEntropy parameter
Token Mask Rate	10%	Applied with 50% probability during training
Dropout	0.3	After pooled output

Submission Checklist

- ☒ Name and student ID included
- ☐ Final write-up exported to PDF
- ☒ All code is in Python and PyTorch or TensorFlow
- ☒ Github repo link included and shared with the POD
- ☒ Performance metrics reported
- ☒ Efficiency metrics for training and inference reported
- ☒ Failure analysis completed
- ☒ Pre-flight / runtime / post-prediction checks documented
- ☒ Stress tests designed and results reported
- ☒ At least two robustness methods implemented and justified
- ☒ At least one adversarial evaluation implemented
- ☒ Robustness curves, confidence histograms, and calibration plots included
- ☒ Clean and robustness metrics reported consistently with Milestone 2
- ☒ Latency / throughput / memory / model size reported
- ☒ 6-10 representative failure examples included
- ☒ At least 2 resolved cases after adaptation included
- ☒ Monitoring plan documented
- ☒ Monitoring notebook or script submitted
- ☒ Adaptation triggers and cadence documented
- ☒ Retraining / update and versioning plan documented
- ☒ Adaptation experiment reported with before/after results
- ☒ Model / training / deployment changes explained and implemented
- ☒ Reproducibility details included